

Final Report: Implementation of a Retrieval-Augmented Generation (RAG) System

1. Overview

In this assignment, I implemented a Retrieval-Augmented Generation (RAG) based conversational chatbot capable of answering user questions based on the content of multiple academic research papers provided in PDF format. The main objective of the system is to retrieve relevant information from documents using semantic similarity and generate accurate, context-grounded responses using a large language model.

The system follows a modular RAG architecture and runs entirely in a local environment. Strict grounding rules were applied to ensure that answers are derived only from retrieved document content and that hallucinations are avoided.

2. System Architecture

The RAG system consists of the following components:

- PDF ingestion and text extraction
- Text cleaning and chunking
- Embedding generation
- Vector database storage using FAISS
- Semantic retrieval and re-ranking
- Answer generation using a local LLM
- Conversational memory
- Streamlit-based user interface

3. Technical Implementation

3.1 Libraries and Tools Used

The following libraries were used in this project:

PyMuPDF (fitz) for PDF text extraction

LangChain for prompt handling, chaining, and memory

SentenceTransformers with all-mpnet-base-v2 for embeddings

FAISS for vector similarity search

Ollama (Llama-3) as the local language model

Streamlit for the user interface

NumPy and Pandas for numerical processing

All configuration values such as chunk size, overlap, embedding model, PDF directory path, vector store path, and LLM model name are maintained in config.py.

3.2 PDF Processing

PDF documents are processed using PyMuPDF. Additional text cleaning is applied to remove broken line endings and join hyphenated words to preserve sentence integrity.

3.3 Chunking Strategy

Text is chunked using LangChain's RecursiveCharacterTextSplitter with a chunk size of 200 characters and an overlap of 30 characters. This configuration balances semantic clarity and retrieval precision.

3.4 Embedding Generation

Each chunk is converted into a dense vector embedding using the all-mpnet-base-v2 model. Embeddings are normalized for reliable similarity computation.

3.5 Vector Database

FAISS is used as the vector database to store embeddings and perform similarity searches. It enables fast and efficient local retrieval.

3.6 Retrieval and Answer Generation

At query time, the user question is embedded and matched against the FAISS index. A secondary semantic re-ranking step selects the most relevant chunks, which are passed to the LLM. Llama-3 generates answers strictly from retrieved context.

3.7 Conversational Memory

The chatbot maintains the last four conversations to support follow-up questions. For batch evaluation, memory is reset per query.

3.8 Design Decisions and Iterative Optimization

During the development of the RAG system, multiple design choices were explored and evaluated before finalizing the current configuration. This iterative approach helped identify stability, compatibility, and accuracy trade-offs commonly encountered in real-world RAG implementations.

3.8.1 Chunking Strategy Experiments

Initially, a larger chunk size (approximately 350 characters with higher overlap) was used to preserve broader context within each document segment. While this approach reduced the total number of chunks, it resulted in less precise retrieval, as individual chunks often contained multiple unrelated concepts from research papers.

To improve retrieval granularity, smaller chunk sizes were later explored. After experimentation, a configuration of 200 characters with 30 characters overlap was finalized. This combination achieved a balanced trade-off between semantic coherence and retrieval precision. The overlap ensured that important contextual information spanning chunk boundaries was preserved while preventing excessive redundancy.

This finalized chunking strategy significantly improved the relevance of retrieved content, especially when dealing with academic PDFs containing dense technical explanations.

3.8.2 Embedding Model Exploration

Multiple embedding options were evaluated during development. Initially, default embeddings provided through the language-model interface were tested. However, these embeddings were found to be less effective in capturing semantic similarity for research-oriented text, particularly when multiple documents discussed related high-level concepts.

After evaluation, the **SentenceTransformers all-mpnet-base-v2 model** was selected as the final embedding model. This model consistently produced high-quality semantic embeddings and demonstrated strong performance in retrieving relevant passages from academic documents. Its robustness and community adoption made it well-suited for a production-style RAG pipeline.

3.8.3 Retrieval Strategy Refinements

The system originally relied on top-K vector retrieval alone. However, when multiple PDFs covering related topics were present, this approach sometimes surfaced relevant but suboptimal chunks.

To address this issue, a two-stage retrieval strategy was introduced:

Initial similarity search using FAISS.

Secondary semantic reranking of the retrieved chunks to select the most contextually relevant content.

This refinement improved answer accuracy without relying on hardcoded logic or documents specific assumptions, maintaining the generality of the RAG system.

3.8.4 Evaluation Tooling Considerations

An attempt was made to incorporate the RAGAS framework for automated evaluation. However, version incompatibility with Python and the latest LangChain releases resulted in persistent integration issues. Rather than

forcing an unstable setup, the decision was made to adopt a custom batch-evaluation approach, ensuring reliability and transparency.

This decision reflects practical engineering considerations often encountered in production environments, where tool compatibility and system stability are prioritized over strict adherence to a particular framework.

3.8.5 Final Configuration Rationale

The final system configuration was selected based on the following criteria:

- Retrieval accuracy across multiple academic PDFs
- Stability of the execution environment
- Clear separation of responsibilities in the codebase
- Strict prevention of hallucination
- Maintainability and ease of extension

The final combination of MPNet embeddings, optimized chunking, FAISS retrieval, re-ranking logic, and Llama-3 generation provided consistent and reliable performance, aligning with real-world expectations for RAG systems.

4. Evaluation Framework

4.1 RAGAS Attempt

An attempt was made to evaluate the system using the RAGAS framework. However, due to incompatibilities between RAGAS, Python versions, and LangChain updates, the framework could not be executed reliably.

4.2 Custom Evaluation

A custom evaluation approach was adopted using a set of predefined questions. Responses were evaluated using `batch_eval.py` based on relevance, accuracy, contextual awareness, and response quality. Answers of 'I don't know' were treated as correct when information was absent.

5. Results and Discussion

The RAG system performs reliably after optimization. Retrieval accuracy improved significantly due to refined chunking, text cleaning, and semantic re-ranking. Avoiding hallucination was a key strength of the final implementation.

6. Conclusion

The project successfully demonstrates a complete RAG pipeline capable of answering questions from multiple PDFs. Despite RAGAS not being usable due to compatibility issues, the custom evaluation framework provided effective validation. The system meets all assignment requirements and demonstrates a strong understanding of RAG principles.

7. How to Run the Application

7.1 Environment Setup

The project was developed using Python 3.11. Create and activate a virtual environment:

```
python -m venv .venv
```

Activate the environment and install dependencies:

```
pip install -r requirements.txt
```

7.2 Prepare Input Documents

Place all PDF files inside the pdfs/ directory. All PDFs in this folder will be ingested automatically.

7.3 Build the Vector Store

Run the following command to build the FAISS index:

```
python -c "from vectorstore import build_faiss_store; build_faiss_store()"
```

7.4 Run the Chatbot

Launch the Streamlit application using:

```
streamlit run rag_app.py
```

7.5 Batch Evaluation

To run batch evaluation, execute:

```
python batch_eval.py
```